

Statistical Machine Learning

Assignment 4 Report

May 4, 2026

Contents

1	Question 1: AdaBoost Classifier	2
1.1	Methodology and Code Walkthrough	2
1.2	Terminal Output	2
1.3	Results and Discussion	2
2	Question 2: Gradient Boosting Regressor	4
2.1	Methodology and Code Walkthrough	4
2.2	Terminal Output and Learning Rate Analysis	4
2.3	Results and Discussion	4
3	Question 3: Rosenblatt Perceptron	7
3.1	Methodology and Code Walkthrough	7
3.2	Results and Discussion	7

1 Question 1: AdaBoost Classifier

1.1 Methodology and Code Walkthrough

In this question, we implemented AdaBoost using Decision Stumps to classify digits (4 vs 9) from the MNIST dataset. The preprocessing step extracted the required classes, replaced the labels with $\{-1, 1\}$, and performed a 70-30 train-test split. Furthermore, PCA was applied to reduce the dimensionality to $p = 5$ components to simplify the learning process and reduce noise.

Our `DecisionStump` class finds the optimal threshold across all features by calculating the weighted error for every possible midpoint between unique values. The main `train_adaboost` loop maintains a set of sample weights $\mathbf{w}$ which are updated iteratively based on the predictions. The amount of say a stump has, α , is calculated based on its weighted error:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{err}_m}{\text{err}_m} \right) \quad (1)$$

1.2 Terminal Output

The script incrementally calculated the test accuracy to give us an understanding of how performance changes over iterations:

Trees	Val Acc	Test Acc

50	0.8090	0.7961
100	0.8090	0.7966
150	0.8085	0.7956
200	0.8095	0.7971
250	0.8095	0.7976
300	0.8095	0.7956

Best Iter: 123 | Val Acc: 0.8120 | Test Acc: 0.7976

1.3 Results and Discussion

The validation accuracy peaks at **iteration 123** with 81.20% accuracy. The corresponding test accuracy is 79.76%. As the number of trees increases past this point, the validation accuracy starts to plateau and dip slightly, showcasing the onset of overfitting, though AdaBoost is generally quite robust to overfitting.

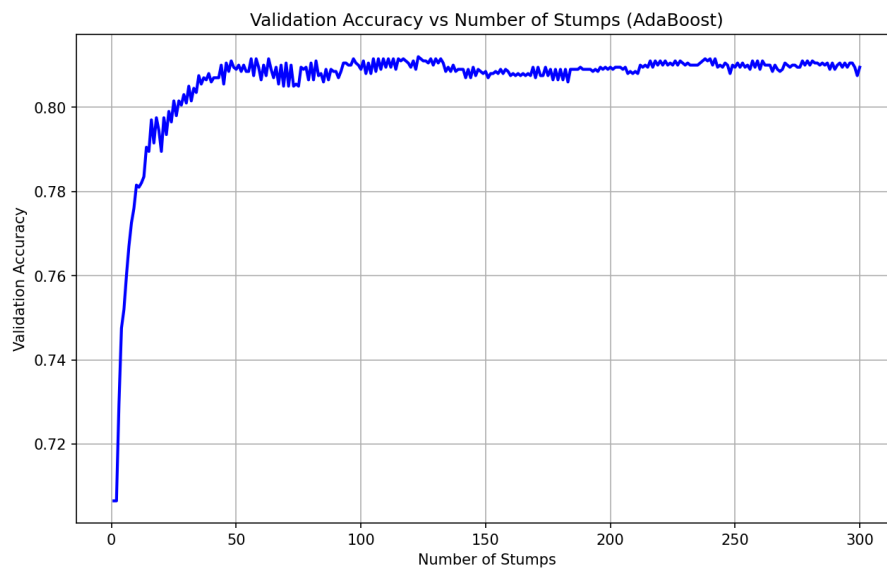


Figure 1: Validation Accuracy vs Number of Trees for AdaBoost

2 Question 2: Gradient Boosting Regressor

2.1 Methodology and Code Walkthrough

For the second question, we utilized a Gradient Boosting Model (GBM) with Decision Stump regressors. Rather than updating sample weights like AdaBoost, Gradient Boosting trains each subsequent model on the **pseudo-residuals** (negative gradient of the loss function) of the aggregated ensemble.

The `DecisionStumpRegressor` evaluates all possible thresholds, minimizing the Sum of Squared Residuals (SSR) efficiently using a vectorized approach. We then evaluate the model over 300 iterations across various learning rates (η).

2.2 Terminal Output and Learning Rate Analysis

The following terminal output illustrates the Train, Validation, and Test Mean Squared Error (MSE) for various values of η :

eta	Best Iter	Train MSE	Val MSE	Test MSE
0.001	300	0.9317	0.9215	0.9290
0.01	115	0.8686	0.8268	0.8572
0.1	12	0.8711	0.8277	0.8586
0.2	6	0.8711	0.8277	0.8586
0.5	2	0.8660	0.8301	0.8555
1	1	0.8662	0.8310	0.8555

2.3 Results and Discussion

To analyze the effect of the learning rate, we compare the global convergence across all values of η and specifically examine the performance of the optimal learning rate.

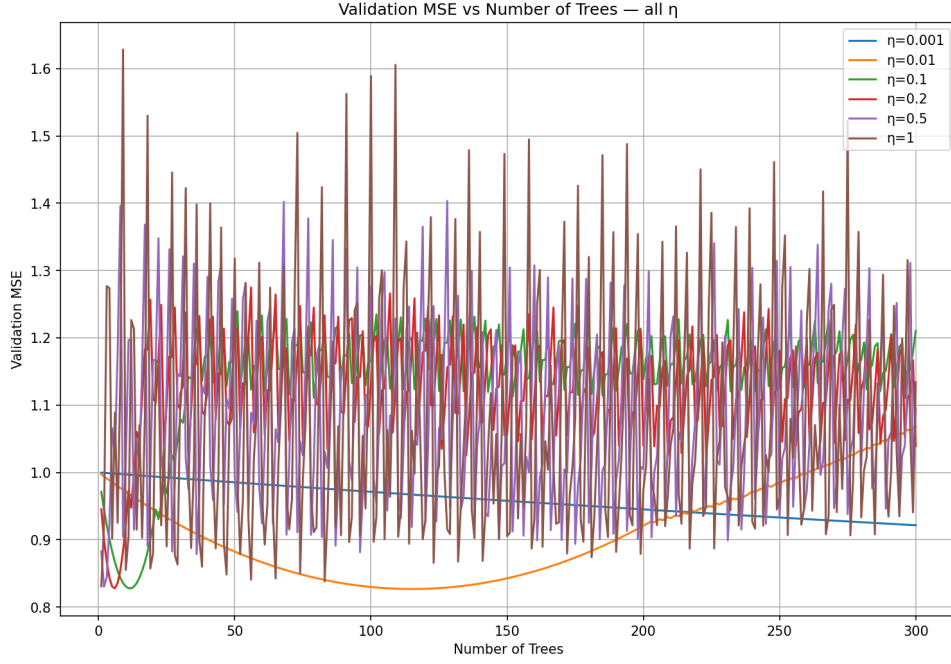


Figure 2: Validation MSE vs Number of Trees for different learning rates

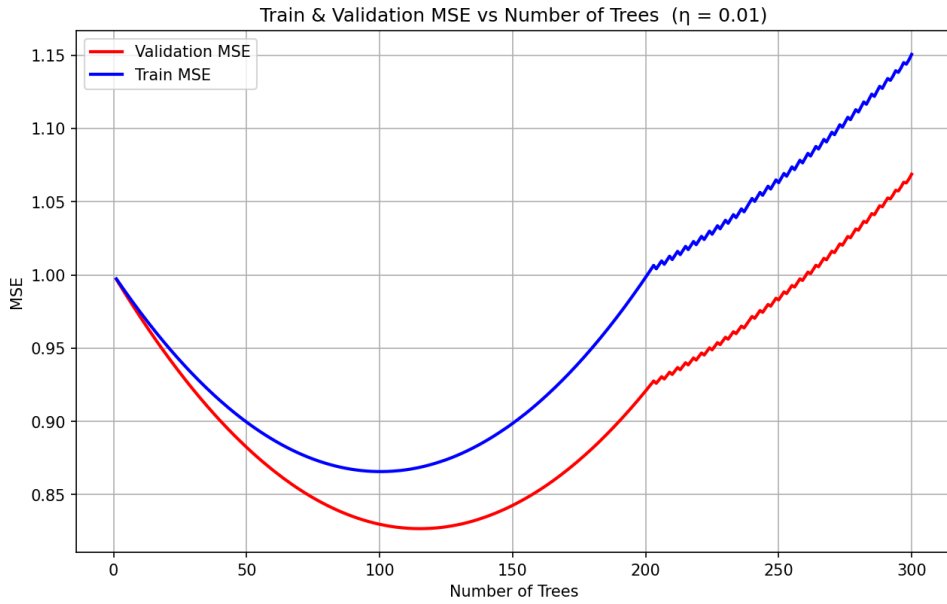


Figure 3: Detailed MSE convergence for the optimal learning rate ($\eta = 0.01$)

Trends and Patterns Observed:

- **Small Learning Rates** ($\eta = 0.001, 0.01$): The model learns slowly. For $\eta = 0.001$, even 300 trees are not enough to reach the optimal minimum (best iteration is 300, meaning it is still underfitting). For $\eta = 0.01$, it finds the sweet spot at 115 trees.
- **High Learning Rates** ($\eta \geq 0.1$): The model converges incredibly fast (in 12, 6, 2, or even 1 iteration). However, high learning rates often cause the algorithm to overshoot the optimal weights, leading to a higher minimum validation MSE compared to a well-tuned smaller learning rate.

Overall, $\eta = 0.01$ provides the best balance, successfully minimizing the validation MSE to ≈ 0.8268 without overshooting.

3 Question 3: Rosenblatt Perceptron

3.1 Methodology and Code Walkthrough

We implemented a Rosenblatt Perceptron from scratch to separate two distinct Gaussian datasets:

- **Dataset A:** Means at $(-3, -3)$ and $(3, 3)$, Covariance $= I$.
- **Dataset B:** Means at $(-3, -3)$ and $(3, 3)$, Covariance $= 3I$.

The perceptron iteratively updates its weights $\mathbf{w}$ using stochastic gradient descent. The update rule is:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i \quad \text{and} \quad b \leftarrow b + \eta y_i \quad (2)$$

3.2 Results and Discussion

The mathematical distance between the means is ≈ 8.48 . For Dataset A (SD=1), the classes are extremely far apart. The perceptron achieves 100% accuracy in just 2 epochs. For Dataset B (SD ≈ 1.73), the increased spread causes points to lie near the boundary, requiring 4 epochs to converge with 98.33% accuracy.

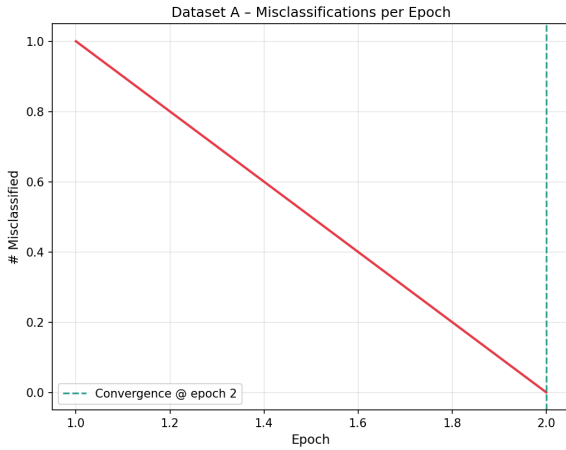


Figure 4: Misclassifications (A)

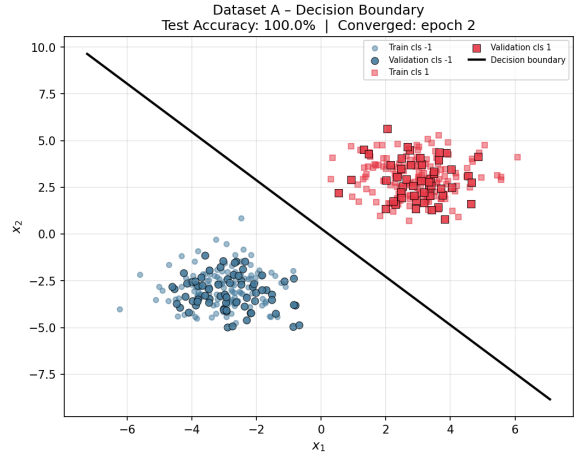


Figure 5: Boundary (A)

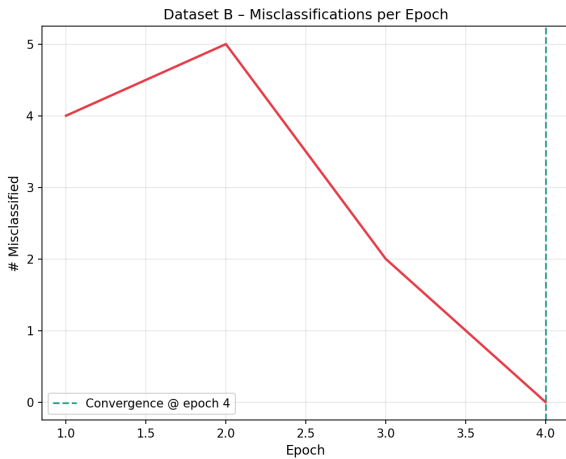


Figure 6: Misclassifications (B)

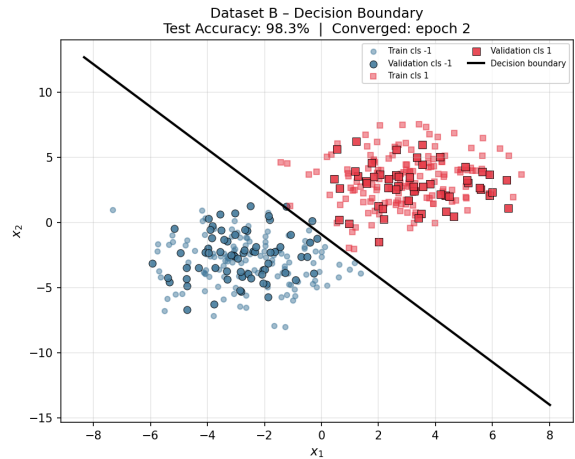


Figure 7: Boundary (B)